

状压DP&树形DP

清华大学 张华清

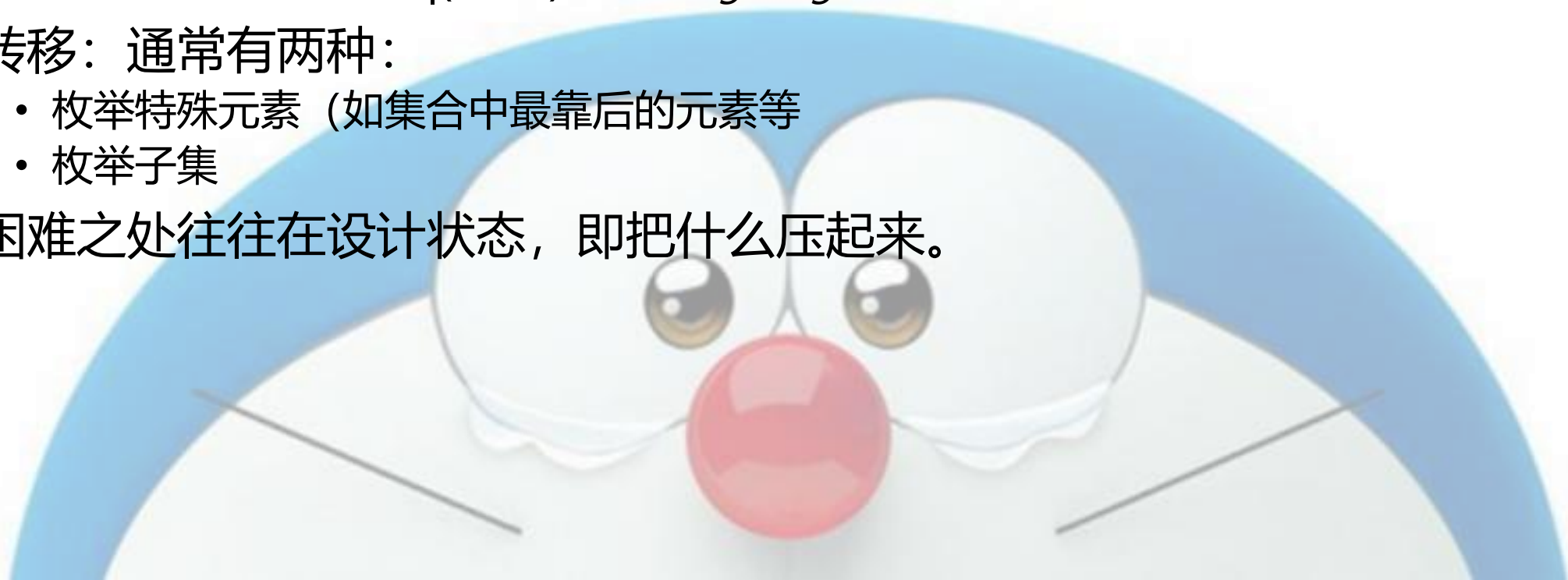


状压DP



状压DP

- 状态是一个集合，使用二进制数把集合压缩为整数。
- 位运算：
 - 提取第 i 位： $(S \gg i) \& 1$
 - 把第 i 位从0变成1： $S | (1 \ll i)$ 若为 *long long*，则需要写成 $1ll \ll i$
- 转移：通常有两种：
 - 枚举特殊元素（如集合中最靠后的元素等
 - 枚举子集
- 困难之处往往在设计状态，即把什么压起来。



状压DP

- 枚举子集：给定集合 S ，遍历其所有子集 $T \subset S$:

```
for(int T=S;;T=(T-1)&S){  
    //T is a subset of S  
}  
//对T=\emptyset特殊处理
```

- 正确性：每次把 T 的最后一个1变成0，并把其右边的能为1的位置都变成1
- 举例：11010->11000->10010->10000->01010->01000->00010->00000
- 复杂度：如下代码的复杂度为 3^n 。考虑每个元素只有三种情况：在 T 内，在 S 内不在 T 内，不在 S 内。

```
for(int S=0;S<(1<<n);S++){  
    for(int T=S;;T=(T-1)&S){  
        //T is a subset of S  
    }  
}
```

Luogu 3694 邦邦的大合唱站队

题目描述

N 个偶像排成一列，他们来自 M 个不同的乐队。每个团队至少有一个偶像。

现在要求重新安排队列，使来自同一乐队的偶像连续的站在一起。重新安排的办法是，让若干偶像出列（剩下的偶像不动），然后让出列的偶像一个个归队到原来的空位，归队的位置任意。

请问最少让多少偶像出列？

【数据规模】

对于20%的数据， $N \leq 20, M = 2$

对于40%的数据， $N \leq 100, M \leq 4$

对于70%的数据， $N \leq 2000, M \leq 10$

对于全部数据， $1 \leq N \leq 10^5, M \leq 20$

Luogu 3694 邦邦的大合唱站队

- $m \leq 10$: $m!$ 枚枚举所有排列方式即可。



Luogu 3694 邦邦的大合唱站队

- $m \leq 20$: 状压DP。用 $f[S]$ 表示 S 集合内的组合是前 $|S|$ 个被选中的。
- 转移考虑 S 中的哪个组合是第 $|S|$ 个被选中的。
- 用前缀和计算代价。



Luogu 3694 邦邦的大合唱站队

- 若一个问题可以用枚举全排列的方法暴搜，而“内部顺序”无后效性，则可以使用状压DP优化。
- 本题中，我们只关心前 $|S|$ 个组合是谁，不关心它们内部是怎么排的。
- 类似地，在TSP问题中，我们只关心走过了哪些点，不关心它们内部走的顺序。需要额外记录最后一步在哪个点。



NOIP2017 宝藏

题目描述

[M+](#) 复制Markdown [🔍](#) 展开

参与考古挖掘的小明得到了一份藏宝图，藏宝图上标出了 n 个深埋在地下的宝藏屋，也给出了这 n 个宝藏屋之间可供开发的 m 条道路和它们的长度。

小明决心亲自前往挖掘所有宝藏屋中的宝藏。但是，每个宝藏屋距离地面都很远，也就是说，从地面打通一条到某个宝藏屋的道路是很困难的，而开发宝藏屋之间的道路则相对容易很多。

小明的决心感动了考古挖掘的赞助商，赞助商决定免费赞助他打通一条从地面到某个宝藏屋的通道，通往哪个宝藏屋则由小明来决定。

在此基础上，小明还需要考虑如何开凿宝藏屋之间的道路。已经开凿出的道路可以任意通行不消耗代价。每开凿出一条新道路，小明就会与考古队一起挖掘出由该条道路所能到达的宝藏屋的宝藏。另外，小明不想开发无用道路，即两个已经被挖掘过的宝藏屋之间的道路无需再开发。

新开发一条道路的代价是 $L \times K$ 。其中 L 代表这条道路的长度， K 代表从赞助商帮你打通的宝藏屋到这条道路起点的宝藏屋所经过的宝藏屋的数量（包括赞助商帮你打通的宝藏屋和这条道路起点的宝藏屋）。

请你编写程序为小明选定由赞助商打通的宝藏屋和之后开凿的道路，使得工程总代价最小，并输出这个最小值。

对于 100% 的数据： $1 \leq n \leq 12$, $0 \leq m \leq 10^3$, $v \leq 5 \times 10^5$ 。

NOIP2017 宝藏

- 题意：求一棵生成树，最小化代价和。其中每条边的代价是长度*深度。



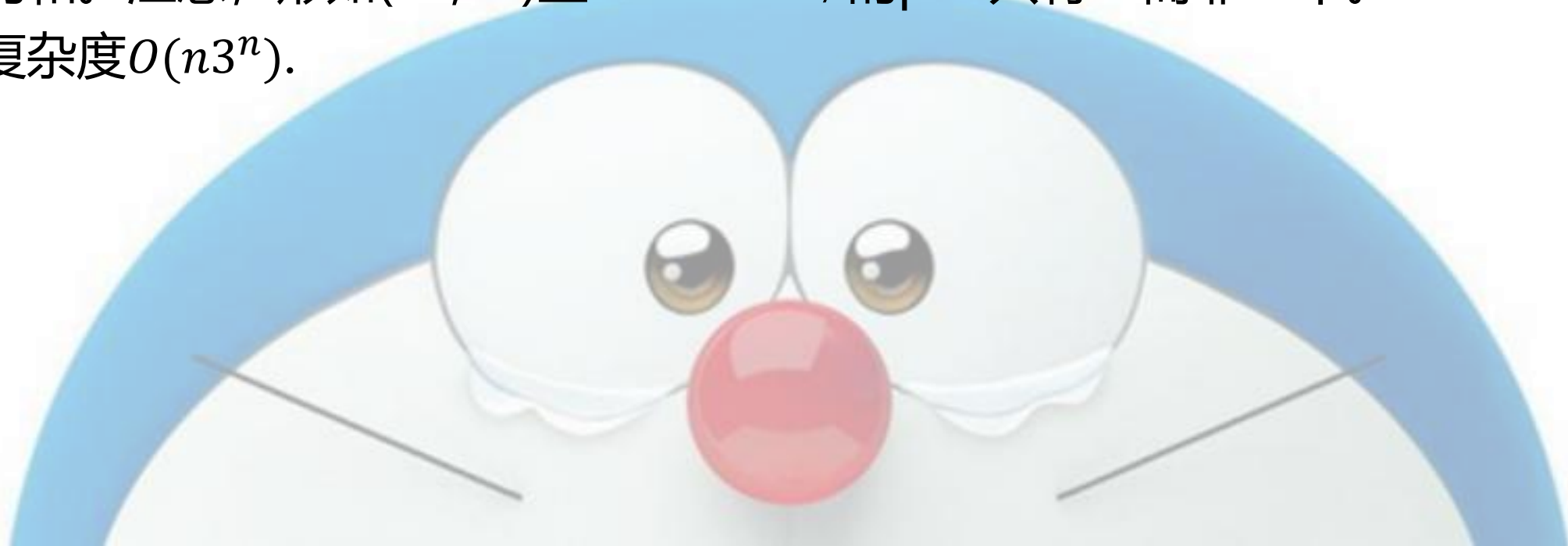
NOIP2017 宝藏

- 看到数据范围显然想状压。
- 我们需要记录什么东西？
 - ✓ 哪些点已被加入树中。
 - ✓ 深度
- 分别记录每个点的深度，一个一个点地加入？
- 记录最深深度，一层一层地加入！



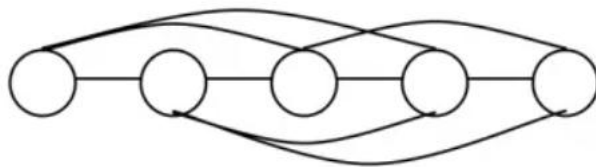
NOIP2017 宝藏

- 设 $f[i][S]$ 为到第 i 层，前 i 层把 S 里的点加入了生成树中。
- $$f[i][S] = \min_{T \subset S} (f[i-1][T] + \text{cost}(T, S \setminus T) * (i-1))$$
- 我们还需要预处理 $\text{cost}(T, S \setminus T)$ ，表示 $S \setminus T$ 中每个点到 T 中任意一点最短距离的和。注意，形如 $(S1, S2)$ 且 $S1 \cap S2 = \emptyset$ 的pair只有 3^n 而非 4^n 个。
- 复杂度 $O(n3^n)$ 。



Luogu 2109 [NOI2007] 生成树计数

- 题意：所有点依次排成一条链，任意编号差的绝对值 $\leq k$ 的点连一条边。求生成树个数。



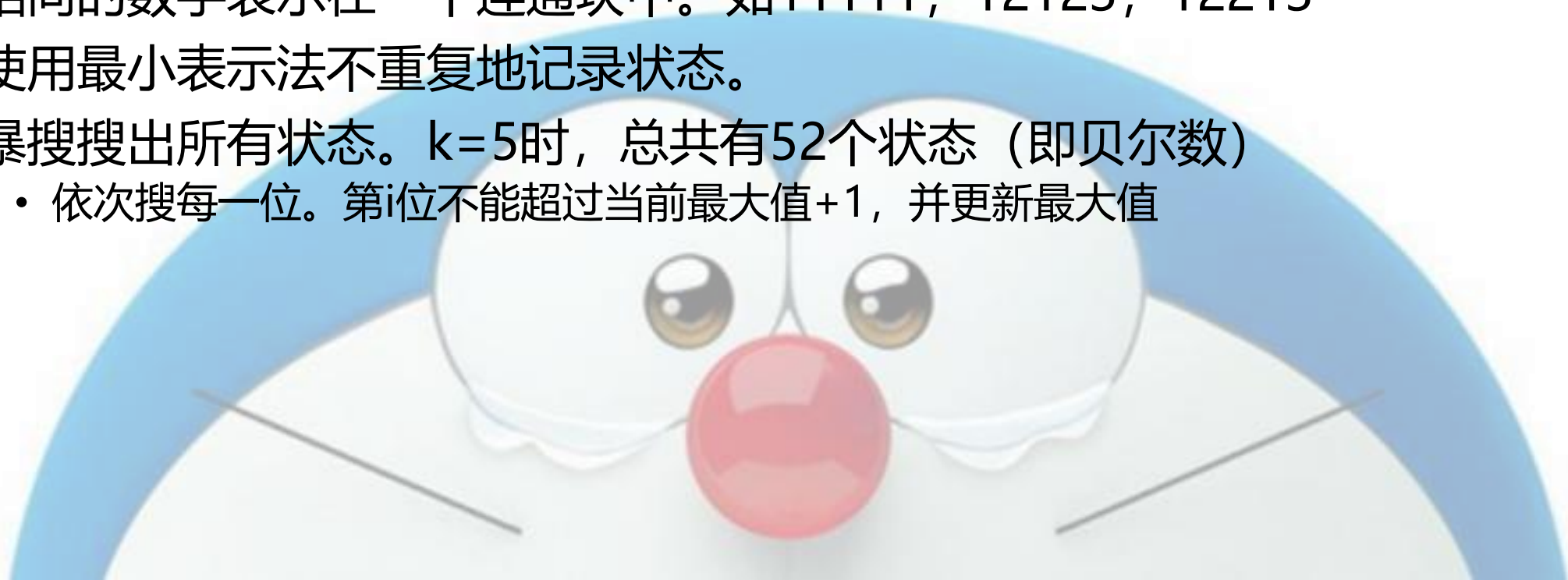
数据规模和约定

对于所有的数据 $2 \leq k \leq n$

数据编号	k 范围	n 范围	数据编号	k 范围	n 范围
1	$k=2$	$n \leq 10$	6	$k \leq 5$	$n \leq 100$
2	$k=3$	$n=5$	7	$k \leq 3$	$n \leq 2000$
3	$k=4$	$n \leq 10$	8	$k \leq 5$	$n \leq 10000$
4	$k=5$	$n=10$	9	$k \leq 3$	$n \leq 10^{15}$
5	$k \leq 3$	$n \leq 100$	10	$k \leq 5$	$n \leq 10^{15}$

Luogu 2109 [NOI2007] 生成树计数

- 与之前的题不同，这个题我们DP状态不是集合，而是记录连通性。
- 更具体地，我们令 $f[i][S]$ 为考虑前 $1\dots i$ 个点内部的边，最后五个点的连通性状态为 S 。
- 相同的数字表示在一个连通块中。如11111, 12123, 12213
- 使用最小表示法不重复地记录状态。
- 暴搜搜出所有状态。 $k=5$ 时，总共有52个状态（即贝尔数）
 - 依次搜每一位。第 i 位不能超过当前最大值+1，并更新最大值



Luogu 2109 [NOI2007] 生成树计数

- 考虑转移。
- 添加 i 号点，我们暴力枚举它和前 k 个点的连边状态。（ 2^k 枚举即可。
- 怎样的连边状态是合法的？
 - 不能成环——用并查集判断
 - 第 $i-k$ 号结点必须和 $[i-k+1, i]$ 内的结点联通起来——否则就再也没机会了！
- 这样，我们就可以枚举出从 $f[i-1][S]$ 转移到 $f[i][S']$ 的系数 $g[S']$ ，即有多少种合法的 i 号点的连边方式，使得 S 状态转移到 S' 状态。
- 接下来显然矩阵快速幂即可。最后答案即 $f[n][11111]$ 。
 - 合法性？全都是联通的。并且自始至终未出现过环。

一个题

- 给定 n 个好串， m 个坏串，求最短的字符串，使其包括所有好串，且不含任意坏串。
- $n \leq 20, m \leq 1e6, |\Omega| = 4$, 串长均为 $k \leq 6$ 。



一个题

- 把所有长为 k 的点拿出来建图。建出转移边。如1234连向2341,2342,2343,2344。
- 删去所有坏点。问题转化成求经过所有好点的最短路径。



一个题

- 设 $f[S][x]$ 为当前走过了 S 中的好点，现在在 x 的最短路径？ $2^{20} * 10^6!$
- ——预处理好点之间两两最短路。求哈密顿路径即可。
- 启发：DP状态较多，考虑只记录关键状态，压缩非关键状态间的转移。



九省联考2018 一双木棋

题目描述

[M+](#) 复制Markdown [🔍](#) 展开

菲菲和牛牛在一块 n 行 m 列的棋盘上下棋，菲菲执黑棋先手，牛牛执白棋后手。

棋局开始时，棋盘上没有任何棋子，两人轮流在格子上落子，直到填满棋盘时结束。

落子的规则是：一个格子可以落子当且仅当这个格子内没有棋子且这个格子的左侧及上方的所有格子内都有棋子。

棋盘的每个格子上，都写有两个非负整数，从上到下第 i 行中从左到右第 j 列的格子上的两个整数记作 $a_{i,j}$ 和 $b_{i,j}$ 。

在游戏结束后，菲菲和牛牛会分别计算自己的得分：菲菲的得分是所有有黑棋的格子上的 $a_{i,j}$ 之和，牛牛的得分是所有有白棋的格子上的 $b_{i,j}$ 的和。

菲菲和牛牛都希望，自己的得分减去对方的得分得到的结果最大。现在他们想知道，在给定的棋盘上，如果双方都采用最优策略且知道对方会采用最优策略，那么，最终的结果如何？

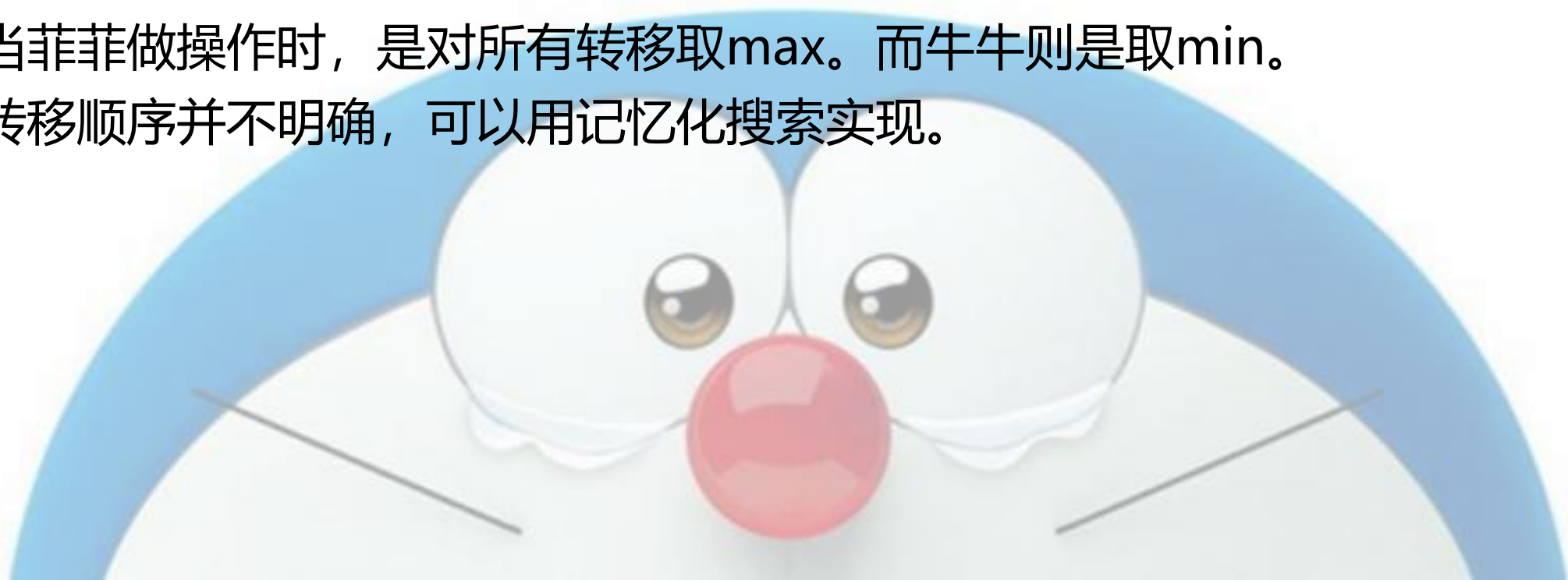
九省联考2018 一双木棋

- 状压的依据：每个人在下棋时，只关心哪些位置被下过了，不关心之前这些棋具体是怎么下的。
- 可以发现被下过棋的位置是一个阶梯型。我们记录轮廓线：从右上角到左下角。0表示向左，1表示向下。



九省联考2018 一双木棋

- 接下来就是经典操作：
- 记录 $f[S]$ 为当前轮廓线为 S 时，从此刻到游戏结束，菲菲得分-牛牛得分会是多少。（注意不是DP从开始到此刻得分是多少。因为每个人做决策时，都是向后看，而不管之前发生了什么）
- 当菲菲做操作时，是对所有转移取 $\max$ 。而牛牛则是取 $\min$ 。
- 转移顺序并不明确，可以用记忆化搜索实现。



Luogu 4547 [THUWC2017] 随机二分图

题目描述

某人在玩一个非常神奇的游戏。这个游戏中有一个左右各 n 个点的二分图，图中的边会按照一定的规律随机出现。

为了描述这些规律，某人将这些边分到若干个组中。每条边或者不属于任何组（这样的边一定不会出现），或者只属于一个组。

有且仅有以下三类边的分组：

0. 这类组每组只有一条边，该条边恰好有 50% 的概率出现。
1. 这类组每组恰好有两条边，这两条边有 50% 的概率同时出现，有 50% 的概率同时不出现。
2. 这类组每组恰好有两条边，这两条边恰好出现一条，各有 50% 的概率出现。

组和组之间边的出现都是完全独立的。

某人现在知道了边的分组和组的种类，想要知道完美匹配数量的期望是多少。你能帮助她解决这个问题吗？

【数据规模和约定】

对于 5% 的数据 $n \leq 5$ 。

对于另 5% 的数据 $n \leq 8$ 。

对于另 10% 的数据 $n \leq 10$ 。

对于另 15% 的数据，只有 $t = 0$ 的情况。

对于另 5% 的数据，只有 $t = 0$ 的情况，且 $m = n^2$ ，也就是该图为一个完全图。

对于另 20% 的数据，只有 $t = 0$ 或者 $t = 1$ 的情况。对于另 20% 的数据，只有 $t = 0$ 或者 $t = 2$ 的情况。对于 100% 的数据， $n \leq 15$ 。

Luogu 4547 [THUWC2017] 随机二分图

- 暴力1：别枚举每种连边状态，然后算最大匹配数。这样复杂度 $O(2^m 2^n)$ 。或许能拿5pts?
- 暴力2：枚举所有可能的最大匹配（也即枚举全排列），算贡献。即多少种情况中，该最大匹配中所有的边都存在。复杂度 $O(n! \cdot n)$ ，期望得分10pts。



Luogu 4547 [THUWC2017] 随机二分图

- 正解沿用了暴力 2^n 算贡献的想法。
- 我们显然还是希望用状压的方法，把 $n!$ 变为 2^n 。比如说，我们想记 $f[S]$ 为左边前 $|S|$ 个结点，匹配了右边 S 集合中的点，所得到的概率和。但困难之处在于，之前一条边的选择与否，会影响同组的边是否被选择。



Luogu 4547 [THUWC2017] 随机二分图

- 有了大致思路后，我们先从简单情况想起。
- 只有0类组？
- 我们设 $dp[S]$ 为，左边前 $|S|$ 个结点匹配右边的 S 集合中结点，能够发生的概率和。
 - 更具体地，它表示：对于所有用前 $|S|$ 个结点匹配 S 中结点的方案，考虑随机生成的图使得该方案合法（也即该方案中所有的匹配边都存在）的概率，的和。
- 转移是简单的：枚举左边第 $|S|$ 号结点匹配谁，乘上 $1/2$ 转移过去即可。
- Q：我们为什么DP概率和而不DP方案数？
- A：概率和只和方案内的边有关，好处理；而方案数和外界有关，不好处理。

Luogu 4547 [THUWC2017] 随机二分图

- 有了1类组和2类组会有什么问题？组内先被DP到的那条边的选择与否，会影响组内第二条边的选择与否——有后效性了。但我们又不能直接把每个组的第一条边是否选择记到DP状态里。
- 同时请注意“在图中”和“是匹配边”是两回事。如第2组边，只是说他们同时在图中，不是说他们同时要成为匹配边。



Luogu 4547 [THUWC2017] 随机二分图

- 先只考虑第0类和第1类边
- 考虑一组匹配方案，它应该贡献 $1/2$ 的几次方的概率和？该方案内有多少**组**边！
- 如果是如果我们就把第1类组里的两条边当成两个独立的边，会发生什么？
- 如果匹配方案中只选中了其中的一条边，那确实就贡献了 $1/2$ ，很合理。
- 如果匹配方案中同时选中了其中的两条边，那就贡献了 $\frac{1}{2}^2$ 的系数，但应该只贡献一个 $\frac{1}{2}$ 。那咋办？容斥掉！额外加入一组转移，即同时选中两条边，其转移系数为 $\frac{1}{4}$ 。那么同时选中两条边的case就会从以上两种方式转移来，其系数的和就是我们想要的 $\frac{1}{2}$ 。

Luogu 4547 [THUWC2017] 随机二分图

- 同理，对于第2类边，如果一个匹配方案选中了两条边中的一条，那系数为 $1/2$ ，没问题。但如果同时选中两个，贡献系数为 $\frac{1}{2^2}$ ，但本应为0！
- 额外加入一组转移，即同时选中两条边，其转移系数为 $-\frac{1}{4}$ 。那么同时选中两条边的case就会从以上两种方式转移来，其系数的和就是我们想要的 $\frac{1}{2}$ 。



Luogu 4547 [THUWC2017] 随机二分图

- 实现细节：需要同时记录左边哪些点匹配了，右边哪些点匹配了。即 $f[S][T]$ 。但总状态数不是 $(2^n)^2$ ，而是 $\sum_{i=0}^n C(n, i)^2 = C(2n, n)$ 。1e8左右，用哈希表存状态。
- 转移时为了不因选边顺序不同重复统计，钦定每次从S集合中编号最小的点出发进行转移。



树形DP



树形DP

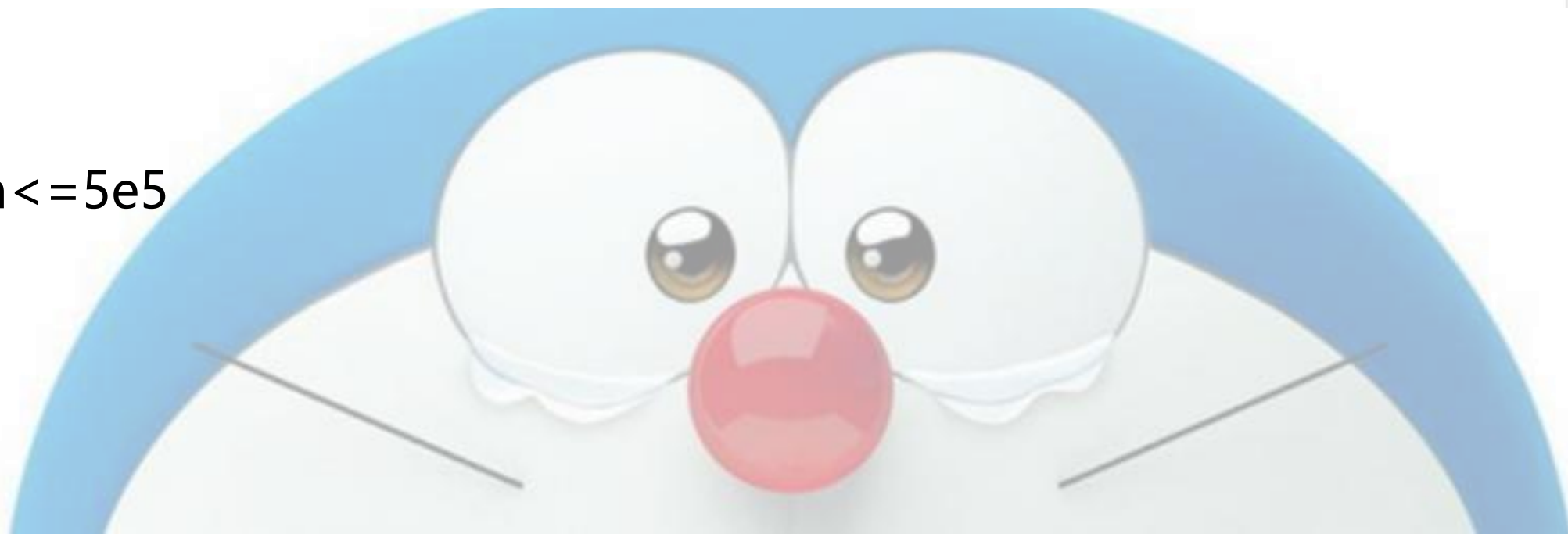
- 在树上做动态规划。
- 往往设 $f[x]$ 为 x 子树内的答案。转移时需要合并子树信息。



2585 [ZJOI2006] 三色二叉树

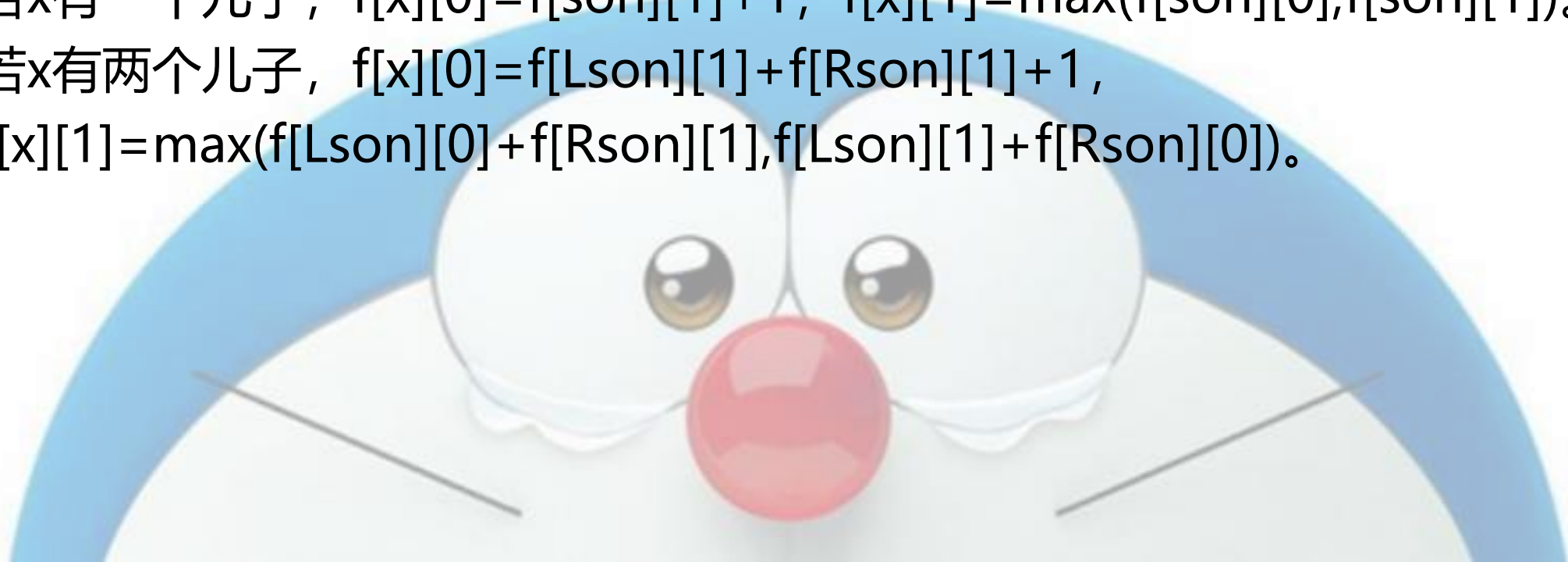
你的任务是要对一棵二叉树的节点进行染色。每个节点可以被染成红色、绿色或蓝色。并且，一个节点与其子节点的颜色必须不同，如果该节点有两个子节点，那么这两个子节点的颜色也必须不同。给定一颗二叉树的二叉树序列，请求出这棵树中**最多和最少**有多少个点能够被染成绿色。

- $n \leq 5e5$



2585 [ZJOI2006] 三色二叉树

- 注意到红色和蓝色地位相等。
- 设 $f[x][0/1]$ 为 x 子树内， x 的颜色为绿色/非绿色时，最多有几个绿色点。
- 若 x 没有儿子， $f[x][0]=1, f[x][1]=0$ 。
- 若 x 有一个儿子， $f[x][0]=f[\text{son}][1]+1, f[x][1]=\max(f[\text{son}][0], f[\text{son}][1])$ 。
- 若 x 有两个儿子， $f[x][0]=f[\text{Lson}][1]+f[\text{Rson}][1]+1,$
- $f[x][1]=\max(f[\text{Lson}][0]+f[\text{Rson}][1], f[\text{Lson}][1]+f[\text{Rson}][0])$ 。



Luogu2014 [CTSC1997] 选课

题目描述

[M+](#) [复制Markdown](#) [\[\]](#) [展开](#)

在大学里每个学生，为了达到一定的学分，必须从很多课程里选择一些课程来学习，在课程里有些课程必须在某些课程之前学习，如高等数学总是在其它课程之前学习。现在有 N 门功课，每门课有个学分，每门课有一门或没有直接先修课（若课程 a 是课程 b 的先修课即只有学完了课程 a ，才能学习课程 b ）。一个学生要从这些课程里选择 M 门课程学习，问他能获得的最大学分是多少？

输入格式

第一行有两个整数 N, M 用空格隔开。 ($1 \leq N \leq 300, 1 \leq M \leq 300$)

接下来的 N 行,第 $I + 1$ 行包含两个整数 k_i 和 s_i , k_i 表示第 I 门课的直接先修课, s_i 表示第 I 门课的学分。若 $k_i = 0$ 表示没有直接先修课 ($1 \leq k_i \leq N, 1 \leq s_i \leq 20$)。



Luogu2014 [CTSC1997] 选课

- 依赖关系显然成一棵有根树
- （其实是有根树森林。不过可以建一个虚拟点当根。



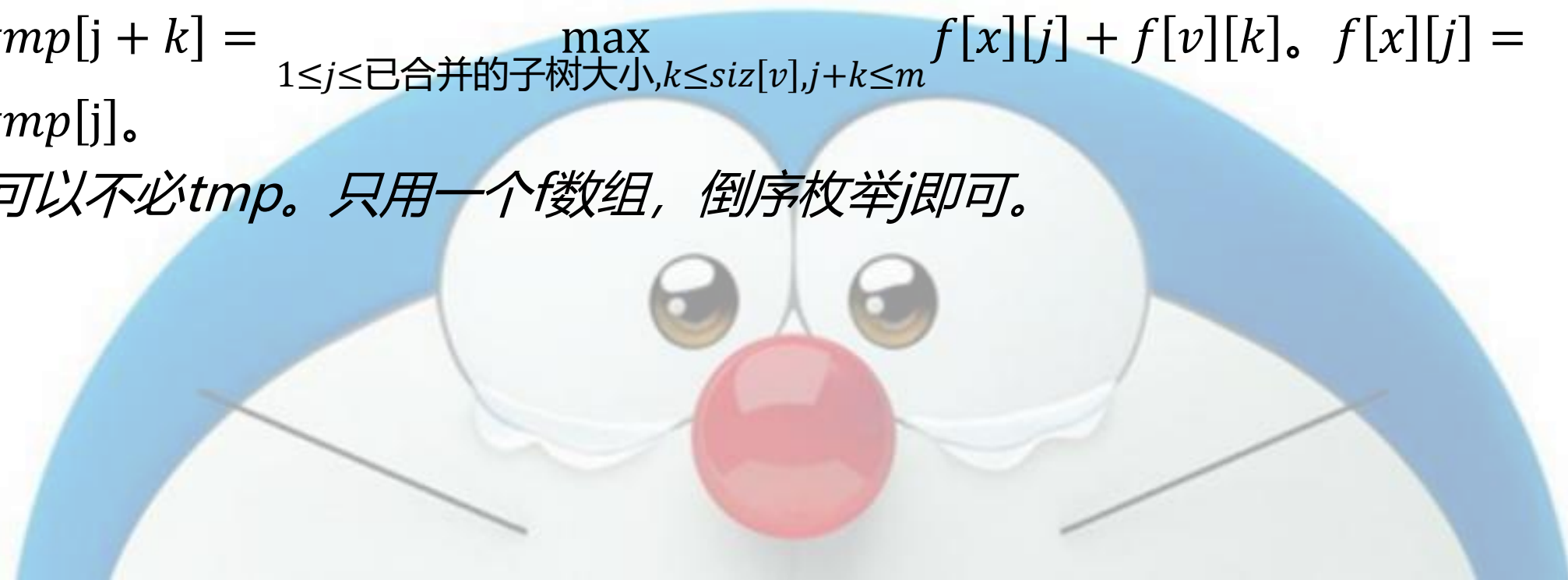
Luogu2014 [CTSC1997] 选课

- 树形背包。
- 普通的背包问题，各个物品之间独立，可以以任意顺序DP。
- 本题中，物品间存在树形的依赖关系。我们不得不用从子树推父亲的特殊顺序进行DP。



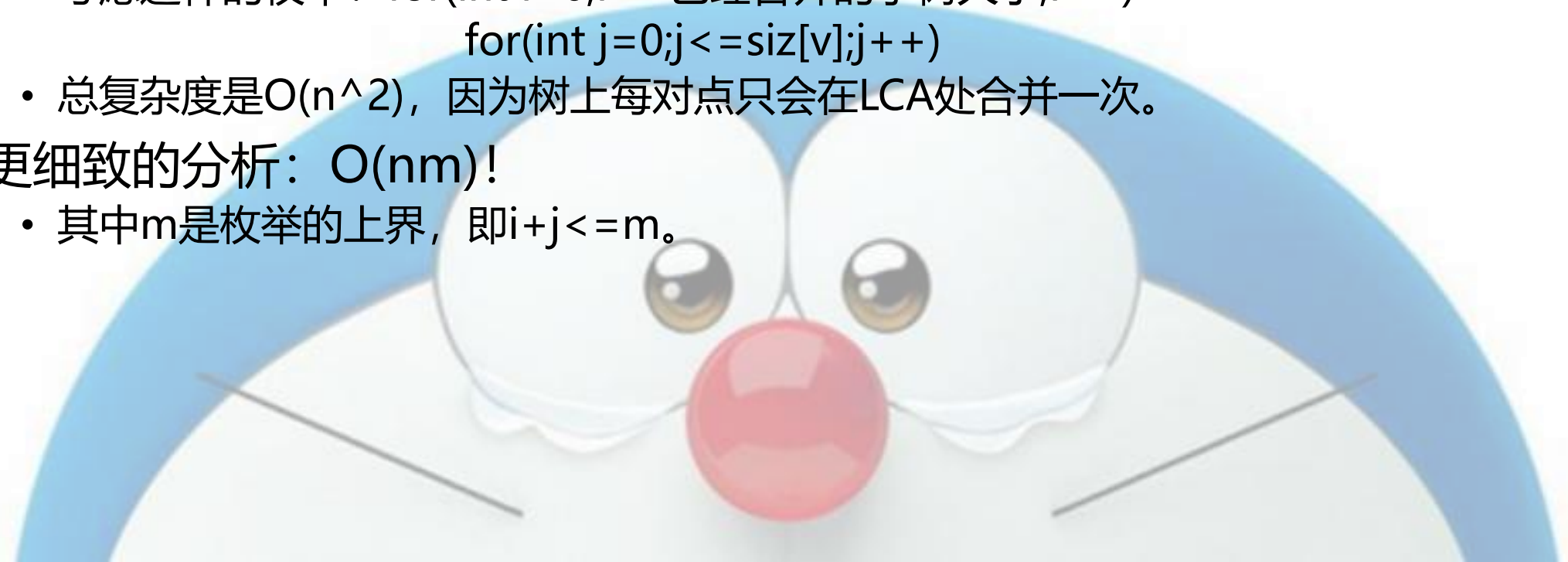
Luogu2014 [CTSC1997] 选课

- 设 $f[x][j]$ 为 x 子树中，已经选择 j 门课程的最大学分。
- 树形DP常见操作：一棵一棵地合并子树。本题中，即为合并背包。
- 初始： $f[x][1] = s[x]$ 。
- $tmp[j + k] = \max_{1 \leq j \leq \text{已合并的子树大小}, k \leq \text{siz}[v], j+k \leq m} f[x][j] + f[v][k]$ 。 $f[x][j] = tmp[j]$ 。
- 可以不必 tmp 。只用一个 f 数组，倒序枚举 j 即可。



Luogu2014 [CTSC1997] 选课

- 复杂度?
- 朴素的分析: $O(nm^2)$ 。
- 较为细致的分析: $O(n^2)$!
 - 考虑这样的枚举: `for(int i=0;i<=已经合并的子树大小;i++)`
`for(int j=0;j<=siz[v];j++)`
 - 总复杂度是 $O(n^2)$, 因为树上每对点只会在LCA处合并一次。
- 更细致的分析: $O(nm)$!
 - 其中 m 是枚举的上界, 即 $i+j \leq m$ 。



4516 [JSOI2018] 潜入行动

题目描述

[M+](#) 复制Markdown [🔍](#) 展开

外星人又双叒要攻打地球了，外星母舰已经向地球航行！这一次，JYY 已经联系好了黄金舰队，打算联合所有 JSOIer 抵御外星人的进攻。

在黄金舰队就位之前，JYY 打算事先了解外星人的进攻计划。现在，携带了监听设备的特工已经秘密潜入了外星人的母舰，准备对外星人的通信实施监听。

外星人的母舰可以看成是一棵 n 个节点、 $n - 1$ 条边的**无向树**，树上的节点用 $1, 2, \dots, n$ 编号。JYY 的特工已经装备了隐形模块，可以在外星人母舰中不受限制地活动，可以神不知鬼不觉地在节点上安装监听设备。

如果在节点 u 上安装监听设备，则 JYY 能够监听与 u **直接相邻**所有的节点的通信。换言之，如果在节点 u 安装监听设备，则对于树中每一条边 (u, v) ，节点 v 都会被监听。特别注意**放置在节点 u 的监听设备并不监听 u 本身的通信**，这是 JYY 特别为了防止外星人察觉部署的战术。

JYY 的特工一共携带了 k 个监听设备，现在 JYY 想知道，有多少种不同的放置监听设备的方法，能够使母舰上**所有节点**的通信都被监听？为了避免浪费，**每个节点至多只能安装一个监听设备，且监听设备必须被用完**。

- $n \leq 1e5, m \leq 100$

4516 [JSOI2018] 潜入行动

- 状态设计
- $f[x][i]$: x 子树内, 放了 i 个监听设备的方案数
- 额外信息?
 - x 点是否被监听了
 - x 点是否有设备
 - 子树内其他点的信息无需被记录。
- $f[x][i][0/1][0/1]$: x 子树内, 放了 i 个监听设备, 其中 x 是否有设备, x 是否被监听的方案数



4516 [JSOI2018] 潜入行动

- 转移:
- $f[x][i+j][0][0] = \sum f[x][i][0][0] * f[v][j][0][1]$: x 没有设备也没被监听, 那么 v 不能有设备并且 v 必须被监听了。
- $f[x][i+j][1][0] = \sum f[x][i][1][0] * (f[v][j][0][0] + f[v][j][0][1])$: x 有设备, 但没被监听, 那么 v 不能有设备。
- $f[x][i+j][0][1] = \sum f[x][i][0][1] * (f[v][j][0][1] + f[v][j][1][1]) + f[x][i][0][0] * f[v][j][1][1]$: x 没设备, 但需要被监听。如果 x 之前已经被监听了, 那么 v 不必有设备, 但 v 自己必须被监听。如果 x 之前没被监听, 那就必须被 v 处的设备监听, 所以 v 必须有设备
- $f[x][i+j][1][1] = \sum f[x][i][1][1] * (f[v][j][0][0] + f[v][j][0][1] + f[v][j][1][0] + f[v][j][1][1]) + f[x][i][1][0] * (f[v][j][1][0] + f[v][j][1][1])$: f 有设备, 也需要被监听。如果之前已经被监听了, 那么对 v 没有任何要求。如果 x 之前没被监听, 那就必须被 v 处的设备监听。
- 说明: 转移方程中, 左边的 $f[x]$ 是合并后的 f , 右边的 $f[x]$ 是合并前的 f 。实现时, 需要新建一个 tmp 数组。

4516 [JSOI2018] 潜入行动

- 状态设计
- $f[x][i]$: x 子树内, 放了 i 个监听设备的方案数
- 额外信息?
 - x 点是否被监听了
 - x 点是否有设备
 - 子树内其他点的信息无需被记录。
- $f[x][i][0/1][0/1]$: x 子树内, 放了 i 个监听设备, 其中 x 是否有设备, x 是否被监听的方案数



CF1324F

题意翻译

- 给定一棵 n 个节点无根树，每个节点 u 有一个颜色 a_u ，若 a_u 为 0 则 u 是黑点，若 a_u 为 1 则 u 是白点。
- 对于每个节点 u ，选出一个**包含** u 的连通子图，设子图中白点个数为 cnt_1 ，黑点个数为 cnt_2 ，请最大化 $cnt_1 - cnt_2$ 。并输出这个值。
- $1 \leq n \leq 2 \times 10^5$, $0 \leq a_u \leq 1$ 。



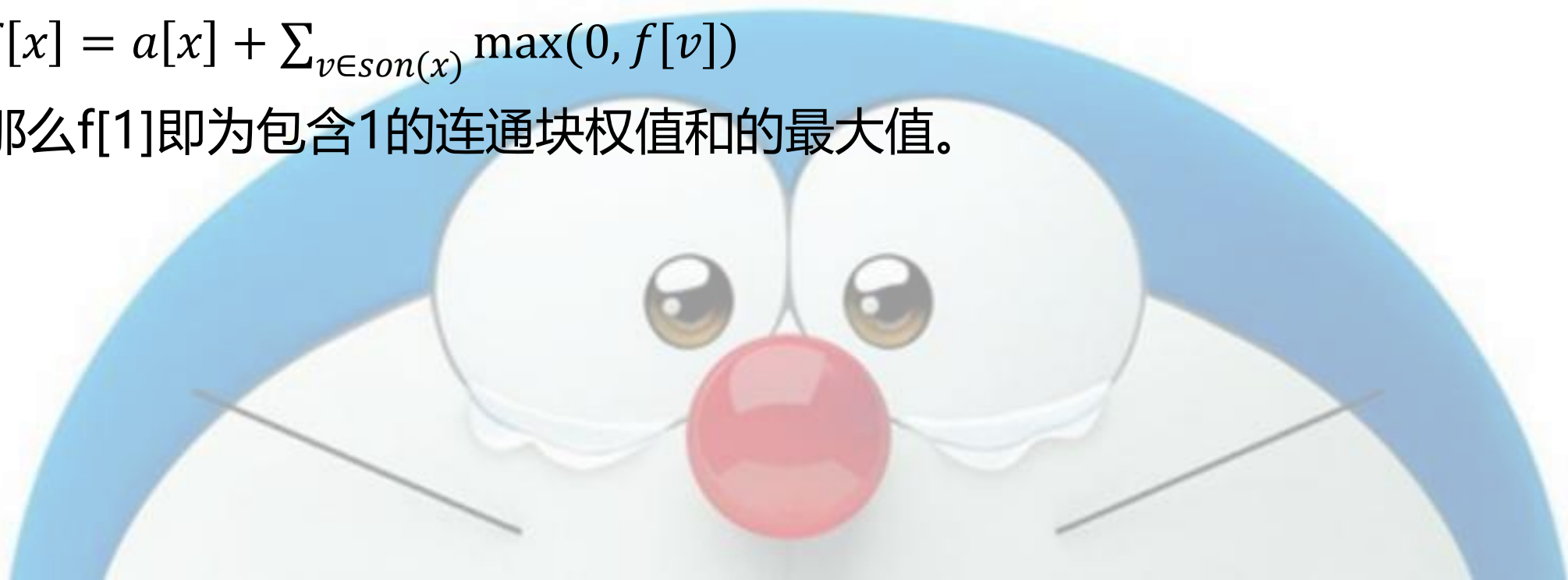
CF1324F

- 换根DP：需要求出以每个点分别作为根时的答案。
- 先随便选取一个点（比如1）为根，进行预处理DP，求出 $f[x]$ 为x子树内的答案。此时 $f[1]$ 即为根节点的答案。
- 接着从根节点出发进行第二次dfs。设 $f'[x]$ 为以x作为根时的答案。
- dfs到x时，x的父亲fa的 f' 值已经求出。我们想由 $f'[fa]$ 和 $f[x]$ 求出 $f'[x]$ 。



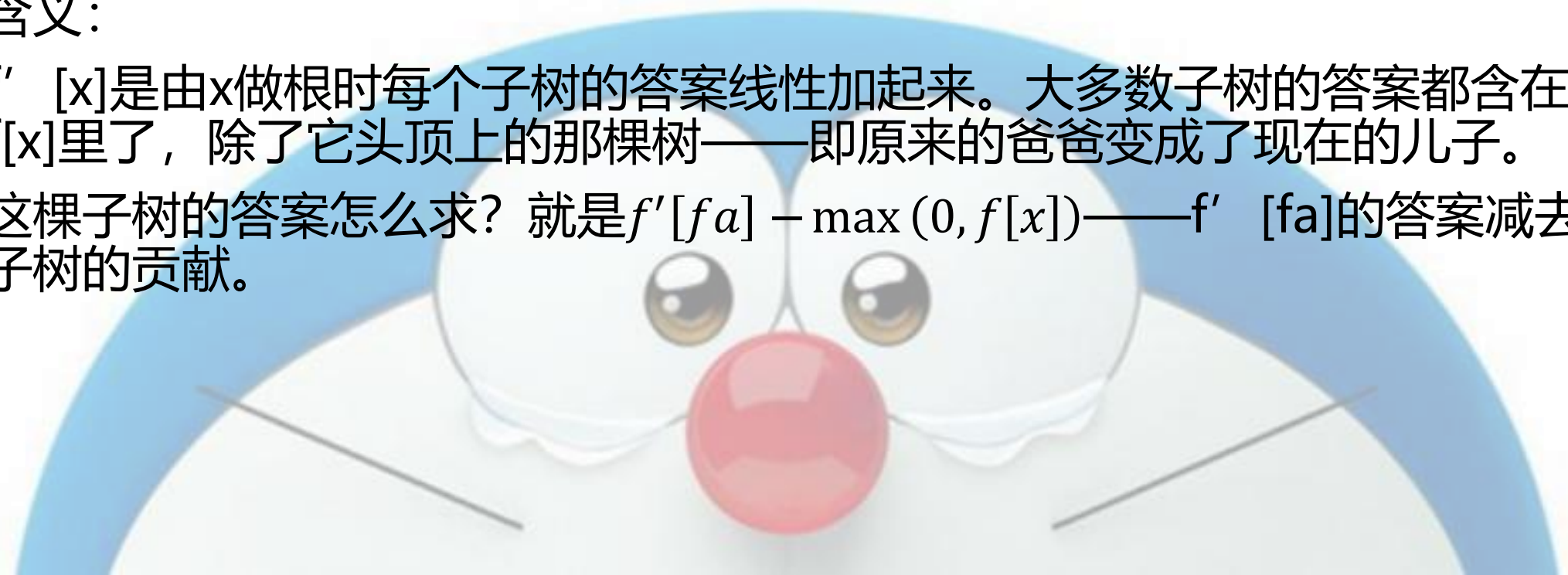
CF1324F

- 回到本题。题意：点带权 ± 1 。对于每个点 x ，求出包含 x 的连通块权值和的最大值。
- 先选定1为根，进行预处理DP：设 $f[x]$ 为 x 子树内，包含 x 的连通块权值和最大值。“包含 x ”是为了保证连通性。
- $f[x] = a[x] + \sum_{v \in \text{son}(x)} \max(0, f[v])$
- 那么 $f[1]$ 即为包含1的连通块权值和的最大值。



CF1324F

- 下面开始第二次dfs（换根）：
- 设 $f'[x]$ 为以 x 为根时的答案， $f'[1]=f[1]$ 。
- $f'[x] = \max(0, f'[fa] - \max(0, f[x])) + f[x]$
- 含义：
- $f'[x]$ 是由 x 做根时每个子树的答案线性加起来。大多数子树的答案都含在 $f[x]$ 里了，除了它头顶上的那棵树——即原来的爸爸变成了现在的儿子。
- 这棵子树的答案怎么求？就是 $f'[fa] - \max(0, f[x])$ —— $f'[fa]$ 的答案减去 x 子树的贡献。



APIO2014 连珠线

题目描述

[M+](#) 复制Markdown [🔍](#) 展开

在达芬奇时代，有一个流行的儿童游戏称为连珠线。当然，这个游戏是关于珠子和线的。线是红色或蓝色的，珠子被编号为 1 到 n 。这个游戏从一个珠子开始，每次会用如下方式添加一个新的珠子：

`Append( $w$ ,  $v$ )`：一个新的珠子 w 和一个已经添加的珠子 v 用红线连接起来。

`Insert( $w$ ,  $u$ ,  $v$ )`：一个新的珠子 w 插入到用红线连起来的两个珠子 u, v 之间。具体过程是删去 u, v 之间红线，分别用蓝线连接 u, w 和 w, v 。

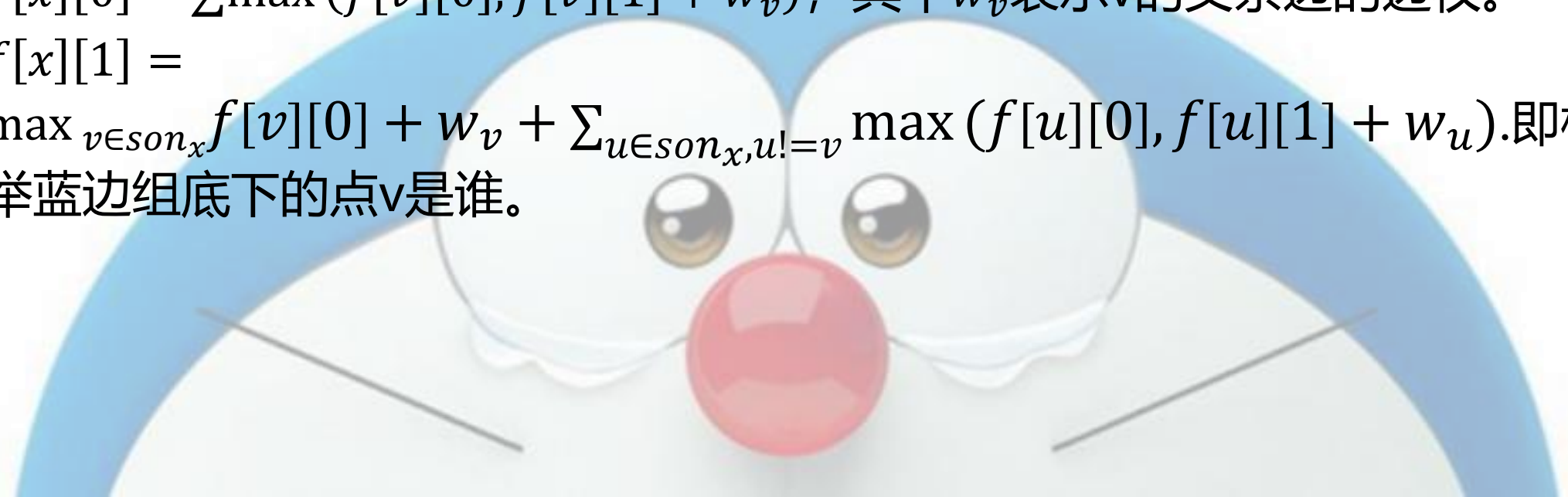
每条线都有一个长度。游戏结束后，你的最终得分为蓝线长度之和。

给你连珠线游戏结束后的游戏局面，只告诉了你珠子和链的连接方式以及每条线的长度，没有告诉你每条线分别是什么颜色。

你需要写一个程序来找出最大可能得分。即，在所有以给出的最终局面结束的连珠线游戏中找出那个得分最大的，然后输出最大可能得分。

APIO2014 连珠线

- 当最开始的唯一结点为根时，所有的蓝色边必须满足：两条蓝边为一组，每一组都是直上直下的。
- 假设1号结点即为最开始的唯一一节点，进行树形DP：
- 设 $f[x][0/1]$ 为 x 子树内， x 是否是某个蓝边组的中间节点。
- $f[x][0] = \sum \max(f[v][0], f[v][1] + w_v)$ ，其中 w_v 表示 v 的父亲边的边权。
- $f[x][1] = \max_{v \in \text{son}_x} f[v][0] + w_v + \sum_{u \in \text{son}_x, u \neq v} \max(f[u][0], f[u][1] + w_u)$.即枚举蓝边组底下的点 v 是谁。



APIO2014 连珠线

- $f[1][0]$ 即为以1结点初始点时的答案。
- 于是我们有一个简单的 $O(n^2)$ 做法，即分别枚举每个点当初始点，进行如上的DP。
- 如何优化？当然还是换根DP。



APIO2014 连珠线

- 我们设 $g[x][0/1]$ 为以 x 为根时， x 原来的爸爸（现在成为了 x 的儿子）的子树内的最大值，当 $fa[x]$ 不是/是某个蓝色组的中间点。
- 进行第二遍dfs。当dfs到 x 时， $g[x][0/1]$ 已经在 $fa[x]$ 处求出。那么以 x 为根的答案即 $f[x][0] + \max(g[x][0], g[x][1] + w_x)$ 。
- 重要任务：求出 x 的各个儿子 v 的 $g[v][0/1]$ 值。



APIO2014 连珠线

- g 的求法和前
- $g[v][0] = f[v][0] + \max(g[v][0], g[v][1])$
- $g[v][1]$ 同样需要
儿子, 或 x 的
 $\max(f[v][0],$
- 预处理最大值
次大值。

```
void dp2(int x, int ff, int fe) {
    ans = max(ans, f[x][0] + max(g[x][0], g[x][1])); // 因为x是根。所以都不能延伸

    int mx = -0x3f3f3f3f, sec = -0x3f3f3f3f, pos = 0; // 第二种转移。要算上根的。记录最大和次大
    int sum = f[x][0] + max(g[x][0], g[x][1]);
    if (sum - max(g[x][0], g[x][1]) + g[x][0] + fe > mx) mx = sum - max(g[x][0], g[x][1]) + g[x][0] + fe, pos = ff;
    for (int i = frm[x]; i; i = edge[i].nxt) {
        int v = edge[i].to; if (v == ff) continue;
        int V = sum - max(f[v][0], f[v][1]) + f[v][0] + edge[i].w;
        if (V > mx) sec = mx, mx = V, pos = v;
        else if (V > sec) sec = V;
    }

    for (int i = frm[x]; i; i = edge[i].nxt) {
        int v = edge[i].to; if (v == ff) continue;
        g[v][0] = f[x][0] - max(f[v][0], f[v][1]) + max(g[x][0], g[x][1]);
        if (v != pos) {
            g[v][1] = mx - max(f[v][0], f[v][1]) + edge[i].w;
        } else g[v][1] = sec - max(f[v][0], f[v][1]) + edge[i].w;
        dp2(v, x, edge[i].w);
    }
}
```

的一个不是 v 的
， -

$g[v]$ ， 否则用

NOI2012 迷失游乐园

数据规模和约定

对于 100% 的数据， $1 \leq W_i \leq 100$ 。

题目描述

 复制Markdown  展开

放假了，小 Z 觉得呆在家里特别无聊，于是决定一个人去游乐园玩。

进入游乐园后，小 Z 看了看游乐园的地图，发现可以将游乐园抽象成有 n 个景点、 m 条道路的无向连通图，且该图中至多有一个环（即 m 只可能等于 n 或者 $n - 1$ ）。

小 Z 现在所在的大门也正好是一个景点。小 Z 不知道什么好玩，于是他决定，从当前位置出发，每次随机去一个和当前景点有道路相连的景点，并且同一个景点不去两次（包括起始景点）。贪玩的小 Z 会一直游玩，直到当前景点的相邻景点都已经访问过为止。

小 Z 所有经过的景点按顺序构成一条非重复路径，他想知道这条路径的期望长度是多少？

小 Z 把游乐园的抽象地图画下来带回了家，可是忘了标哪个点是大门，他只好假设每个景点都可能是大门（即每个景点作为起始点的概率是一样的）。同时，他每次在选择下一个景点时会等概率地随机选择一个还没去过的相邻景点。

n	m	备注
$n = 10$	$m = n - 1$	保证图是链状
$n = 100$	$m = n - 1$	只有节点 1 的度数大于 2
$n = 1000$	$m = n - 1$	/
$n = 10^5$	$m = n - 1$	/
$n = 10^5$	$m = n - 1$	/
$n = 10$	$m = n$	/
$n = 100$	$m = n$	环中节点个数 ≤ 5
$n = 1000$	$m = n$	环中节点个数 ≤ 10
$n = 10^5$	$m = n$	环中节点个数 ≤ 15
10	$n = 10^5$	环中节点个数 ≤ 20

NOI2012 迷失游乐园

- 树的情形?
- 设 $f[x]$ 为从 x 出发，一定走向 x 子树内的条件下的期望长度。
- $f[x] = \frac{1}{d_x - [x \neq 1]} \sum_{v \in \text{son}_x} f[v] + \text{dis}(v, x)$ 。
- 此时 $f[1]$ 即为从1出发的期望长度。换根即可求出从任一点出发的期望长度。



NOI2012 迷失游乐园

- 首先，对于环上每个点为根的小树，进行如上的DP，求出 $f[x]$ 。
- 接着，枚举环上的每一个点 x ，考虑求出从 x 出发的答案。考虑若第一步顺时针走，用走到环上某个点 u 并进入其子树的概率 $f[u]$ 即可。逆时针同理。
- 求出环上每个点的答案后，用同样的方法进入到每棵小树内换根。



4099 [HEOI2013] SAO

- 题意：求树形有向图的拓扑序个数。 $n \leq 1000$



4099 [HEOI2013] SAO

- 先考虑外向树。
- 拓扑序的要求：每个点 x 都是其子树中第一个出现的。
- 各个子树独立。则合法拓扑序个数为 $\frac{n!}{\prod \text{siz}[x]}$ ，其中 $\text{siz}[x]$ 为 x 子树大小。



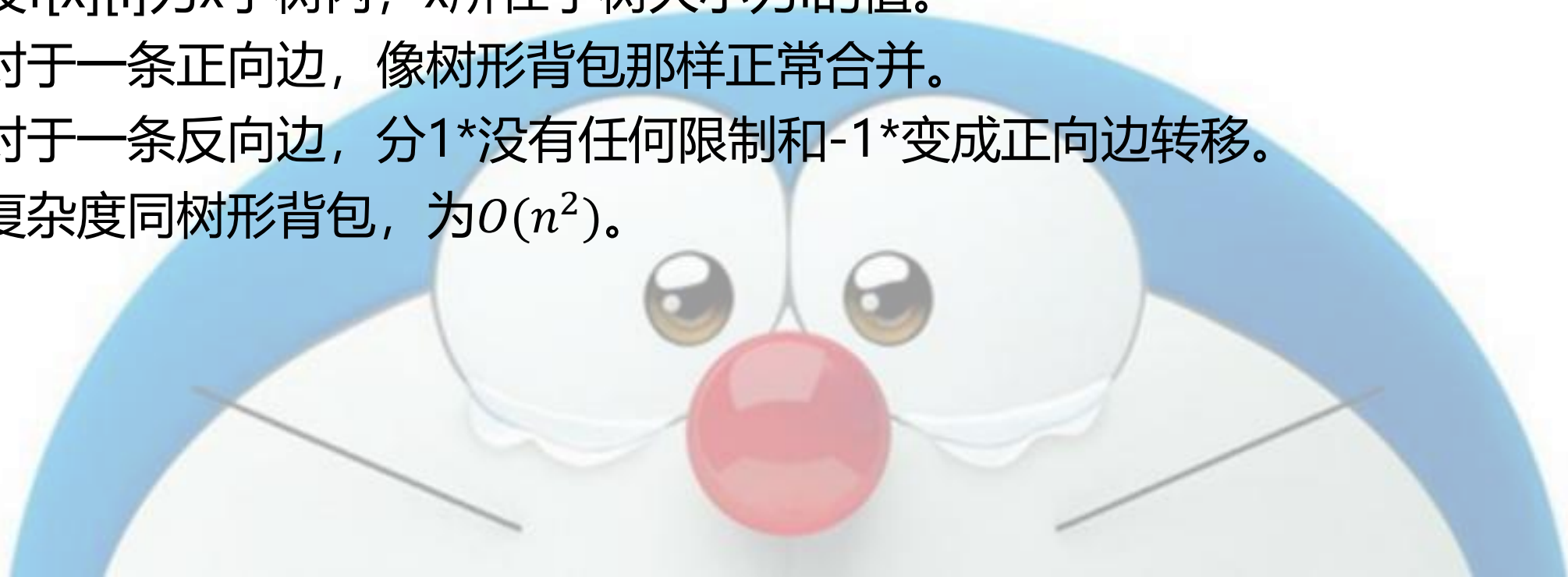
4099 [HEOI2013] SAO

- 再考虑有反向边的情形。
- 有正有反不好做，考虑容斥。
- 把每一条反向边拆成：（没有任何限制-有正向边）。
- 则总方案数为：所有反向边都不存在的方案数-某一条反向边改为正向边，其他反向边不存在+某两条反向边改为正向边，其他反向边不存在+...
- 我们当然可以 2^n 枚举每条反向边是否存在，得到的结果即外向树森林，答案还是 $\frac{n!}{\prod \text{siz}[x]}$ 。



4099 [HEOI2013] SAO

- 容斥DP! ——把系数放到DP过程中。
- 只需要记录所需的额外信息, 本题中即为x所在子树大小 (即把所有 “当作不存在” 的反向边删去后的子树大小) 。
- 设 $f[x][i]$ 为x子树内, x所在子树大小为i的值。
- 对于一条正向边, 像树形背包那样正常合并。
- 对于一条反向边, 分 $1*$ 没有任何限制和 $-1*$ 变成正向边转移。
- 复杂度同树形背包, 为 $O(n^2)$ 。



Luogu 6799 「StOI-2」 独立集

一棵由 n 个点组成的无根树，给定 m 条树上的路径，请求出由这 m 条路径组成的 **独立集** 方案总数。

由于这个答案可能很大，您只需求出它对 998, 244, 353 取模的结果即可。

所谓 **独立集**，就是一个路径集合，满足这个集合中**不存在**一对在树上有交点的路径。特殊的，空集和只包括一条路径的集合也是独立集。

- $n, m \leq 5e5$



Luogu 6799 「StOI-2」 独立集

- 先考虑序列做法。把链按右端点排序。设 $f[i]$ 为考虑了前 i 条链的答案。转移考虑第 i 条链选不选。
- 不选？直接继承 $f[i-1]$ ！
- 选？那就找最靠后的，与 i 不相交的链 pos （可以二分），继承 $f[pos]$ 。
- 即 $f[i] = f[i-1] + f[pos]$



Luogu 6799 「StOI-2」 独立集

- 下面我们试图把如上方法扩展到树上。
- 问题在于序列上每个点只有一个前驱，而树却又多个（多个儿子）
- 如果选择了一条链，序列情形：这段区间所有点都不能选；树上情形：链上的点不能选，但它的子树们可以选。
- 我们不再能设 $f[i]$ 为考虑前 i 条链的答案，而应该设 $f[x]$ 为考虑了 x 子树内的点。



Luogu 6799 「StOI-2」 独立集

- 序列情形，我们选择按右端点对链们排序之后，在右端点考虑每一条链选不选。那树上问题，我们应在哪个位置决定这条链选不选呢？
- ——LCA处！



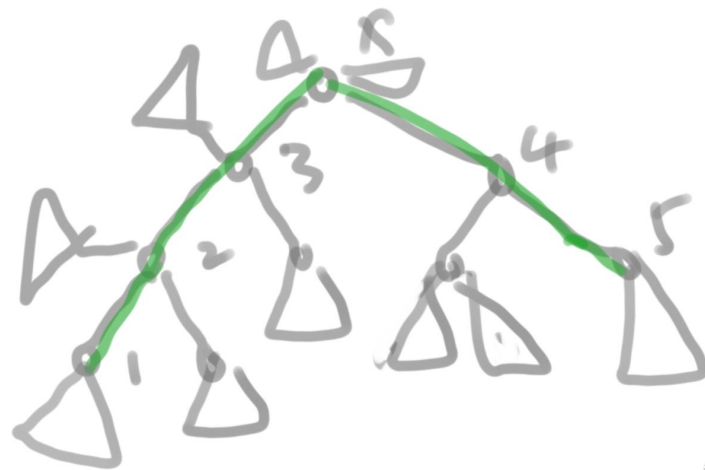
Luogu 6799 「StOI-2」 独立集

- 设 $f[x][0/1]$ 为仅考虑 x 子树内的链的方案数，其中 x （没）有被覆盖。
- 记 $g[x]=f[x][0]+f[x][1]$ ，即总方案数。
- $f[x][0]$? x 自己不被覆盖，它的儿子们随便选。 $f[x][0] = \prod_{v \in \text{son}(x)} g[v]$ 。
- $f[x][1]$? 选且仅能选一条lca为 x 的链。加法原理，依次考虑lca为 x 的那些链选不选。



Luogu 6799 「StOI-2」 独立集

- 画个图：假设选了这个链：我们发现x的大多数儿子都随便选（各个子树之间独立，直接把g乘进去），除了3和4两个在链上的点（不能被覆盖）。
- 那3子树方案数是多少？发现它大多数点也都可以随便选，除了2这个点（不能被覆盖），即 $f[3][0]/g[2]$ 。
- 2子树？ $f[2][0]/g[1]$ 。
- 1子树？它所有儿子随便选，即 $f[1][0]$ 。
- 则本例中，
$$f[x][1] = (f[x][0]/g[3]/g[4]) * (f[3][0]/g[2]) * (f[2][0]/g[1]) * f[1][0] * (f[4][0]/g[5]) * f[5][0]$$
- 推而广之，设 $h[u] = f[u][0]/g[u]$ ，则答案为链上除了x所有点的h值乘起来，再乘上 $f[x][0]$ 。



Luogu 6799 「StOI-2」 独立集

- 接下来就是数据结构问题。求带修树上路径的积？
- 别树剖（两个log）！用树上差分，转为子树修改单点查询。（即，把每个点的权值的含义变为到根路径权值积。）dfs序上建树状数组即可。



Luogu 6799 「StOI-2」 独立集

- 其实，还有一个问题：我们定义 $h[x]=f[x]/g[x]$ ，但 $g[x]$ 可能无逆元。
- DP转移式大概长这样：
$$f[x][1]=(f[x][0]/g[3]/g[4])*(f[3][0]/g[2])*(f[2][0]/g[1])*f[1][0]*(f[4][0]/g[5])*f[5][0]$$
- 我们抛弃原来 h 的定义，转而定义为 $h[x]=f[fa[x]][0]/g[x]=\prod_{v \in son(fa[x]), v \neq x} g[v]$ 。
- $h[x]$ 可以把 $fa[x]$ 的所有儿子拉出来，求一个 g 的前缀和和后缀和，把 x 挖掉即可。
- 但 $f[x][0]/g[3]/g[4]$ 就无法这么做。还得对每个点建线段树求区间和。

Luogu 6799 「StOI-2」 独立集

- 其实，还有一个问题：我们定义 $h[x]=f[x]/g[x]$ ，但 $g[x]$ 可能无逆元。
- DP转移式大概长这样：
$$f[x][1]=(f[x][0]/g[3]/g[4])*(f[3][0]/g[2])*(f[2][0]/g[1])*f[1][0]*(f[4][0]/g[5])*f[5][0]$$
- 我们抛弃原来 h 的定义，转而定义为 $h[x]=f[fa[x]][0]/g[x]=\prod_{v \in son(fa[x]), v \neq x} g[v]$ 。
- $h[x]$ 可以把 $fa[x]$ 的所有儿子拉出来，求一个 g 的前缀和和后缀和，把 x 挖掉即可。
- 但 $f[x][0]/g[3]/g[4]$ 就无法这么做。还得对每个点建线段树求区间和。

5904 [POI2014] HOT-Hotels 加强版

题目描述

给出一棵有 n 个点的树，求有多少组点 (i, j, k) 满足 i, j, k 两两之间的距离都相等。

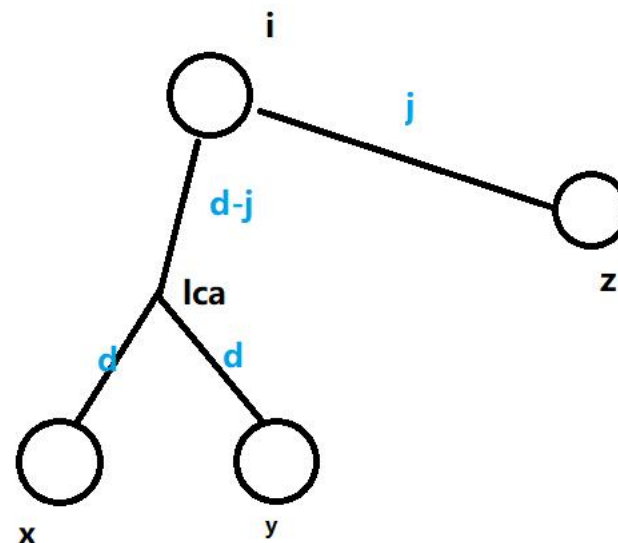
(i, j, k) 与 (i, k, j) 算作同一组。

- $n \leq 1e5$



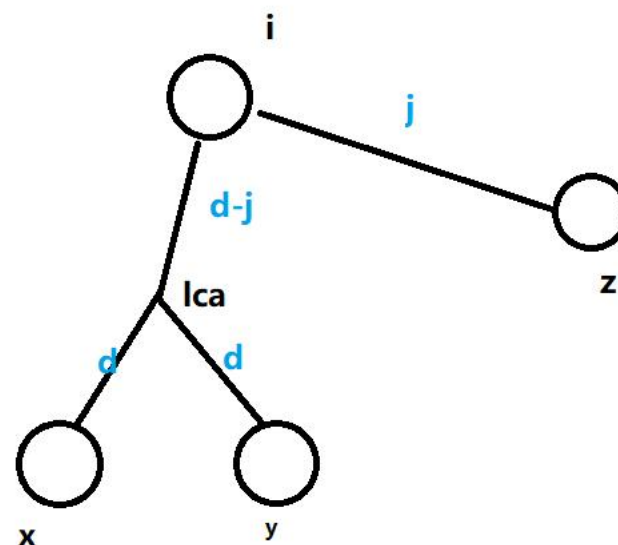
5904 [POI2014] HOT-Hotels 加强版

- 符合条件的 (i,j,k) 只可能是这种情况
- 我们选择在 i 处统计答案。需要记录其子树内有多少对点 (x,y) , 满足 $\text{dis}(x,\text{lca}(x,y)) - \text{dis}(\text{lca}(x,y),i) = \text{dis}(y,\text{lca}(x,y)) - \text{dis}(\text{lca}(x,y),i) = d$



5904 [POI2014] HOT-Hotels 加强版

- 设 $g[u][i]$ 为 u 子树中，这样二元组的个数
- 为了求出 g ，我们需要额外一个数组 $f[u][i]$ 表示 u 子树内，距离 u 为 i 的点的个数。
- $ans += f[x][i] * g[v][j+1]$
- $ans += g[x][i] * f[v][j+1]$
- $g[x][i] += g[v][i+1]$
- $g[x][i] += f[x][i] * f[v][i-1]$
- $f[x][i] += f[v][i-1]$
- 复杂度 $O(n^2)$



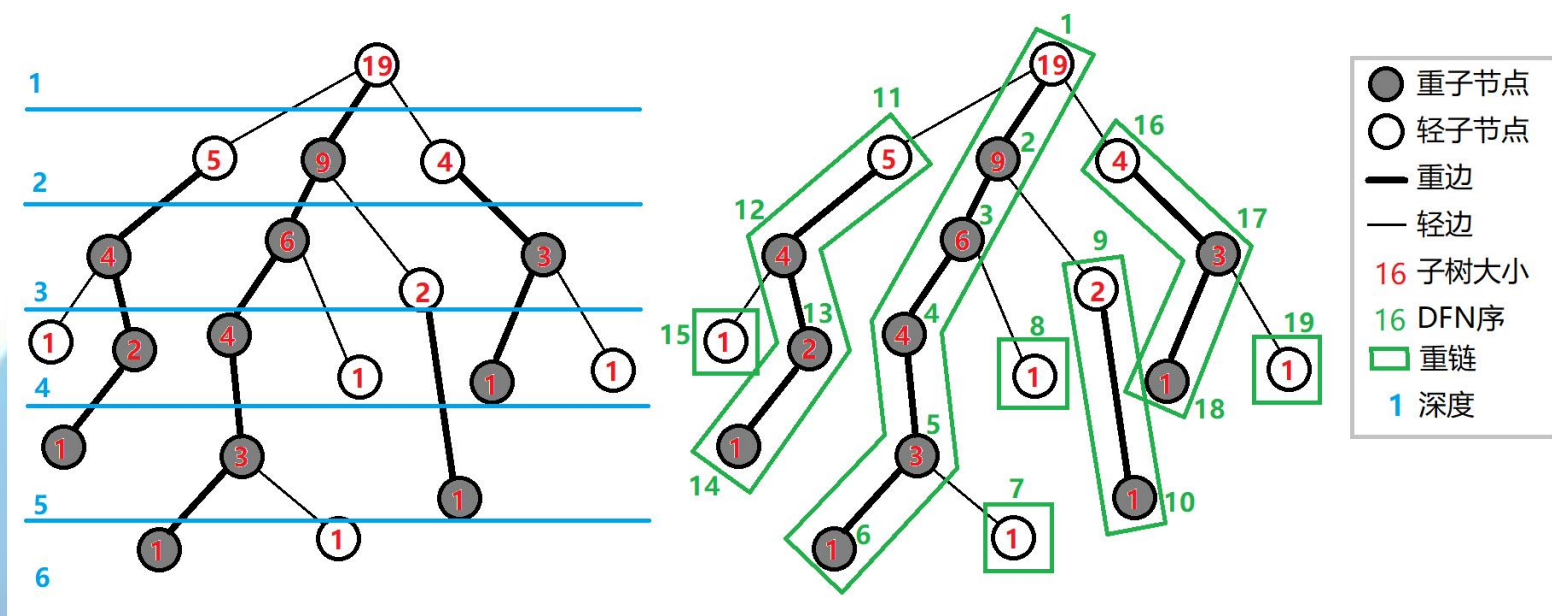
5904 [POI2014] HOT-Hotels 加强版

- 注意到DP数组的第二维和深度有关，且大小是x子树的深度，考虑使用**长链剖分**优化。



长链剖分

- 对于树上的结点 x ，定义其**重儿子**为其子树深度最大的子结点。其他子节点为轻儿子。
- 相应定义**重边**，**轻边**。若干首尾相连的重边称为**重链**。



长链剖分优化DP

- 对于每个点，如果我们能 $O(1)$ 地直接处理其重儿子，再以 $O(\text{lth}[v])$ 的复杂度分别处理其轻儿子 v ，那么整个DP的总复杂度为 $O(n)$ 。其中 $\text{lth}[v]$ 是 v 子树的深度，也即以 v 为顶点的长链长度。



5904 [POI2014] HOT-Hotels 加强版

- $ans += f[x][i] * g[v][j+1]$
- $ans += g[x][i] * f[v][j+1]$
- $g[x][i] += g[v][i+1]$
- $g[x][i] += f[x][i] * f[v][i-1]$
- $f[x][i] += f[v][i-1]$
- 对于本题，如何“ $O(1)$ 地直接处理重儿子”？直接把它f和g数组占为己用！
- 我们先合并重儿子，此时第一、二、四行转移不存在/本身 $O(1)$ 。对于第三行和第五行，直接把重儿子的f/g数组拿过来用就好！
- 对于轻儿子们，正常转移就OK。

5904 [

- 实现上，需最大深度的

```
void dfs(int x,int ff){
    f[x][0]=1;
    if(!mxs[x])return;
    f[mxs[x]]=f[x]+1,g[mxs[x]]=g[x]-1;
    dfs(mxs[x],x);
    ans+=g[mxs[x]][1];

    for(int h=frm[x];h;h=edge[h].nxt){
        int v=edge[h].to;
        if(v==ff||v==mxs[x])continue;
        f[v]=++totf;totf+=mxl[v];totg+=mxl[v],g[v]=++totg,totg+=mxl[v];
        dfs(v,x);
        for(int i=1;i<=mxl[v]+1;i++){
            ans+=1ll*g[x][i]*f[v][i-1];
            if(i!=mxl[v]+1)ans+=1ll*g[v][i]*f[x][i-1];
        }
        for(int i=1;i<=mxl[v]+1;i++){
            if(i!=mxl[v]+1)g[x][i-1]+=g[v][i];
            g[x][i]+=f[x][i]*f[v][i-1];
        }
        for(int i=1;i<=mxl[v]+1;i++)f[x][i]+=f[v][i-1];
    }
}
```

反

两倍自己

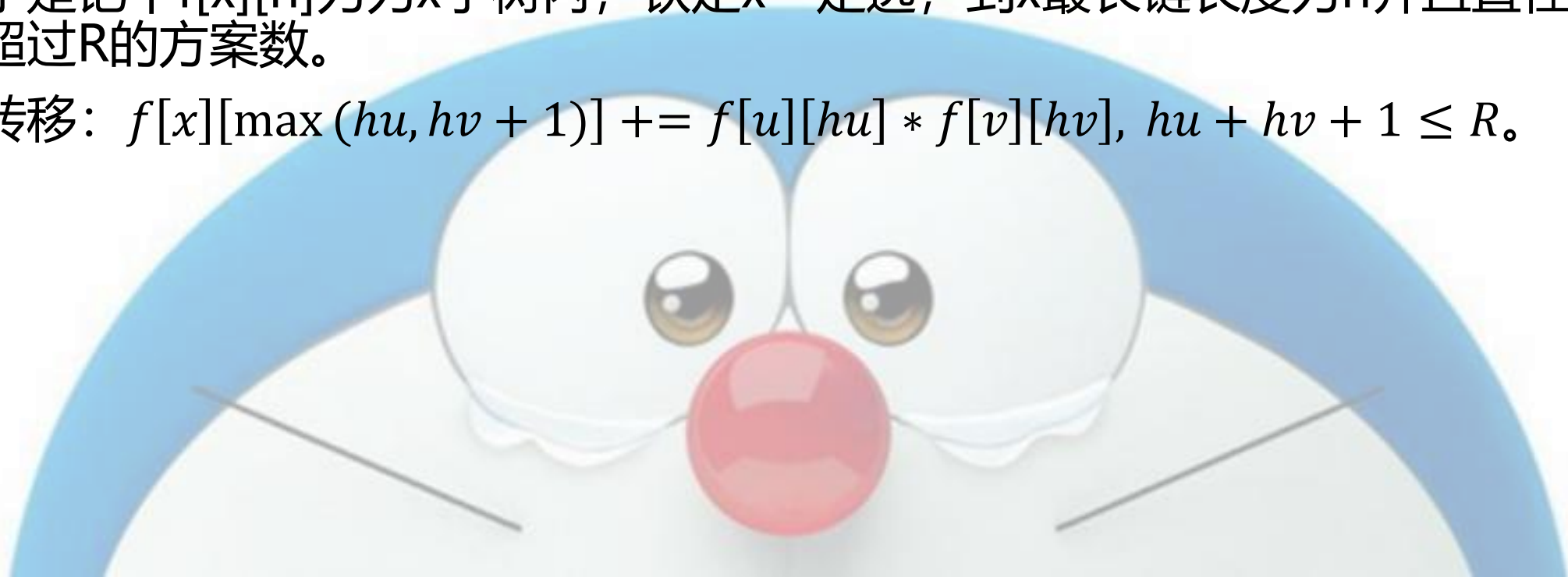
一个题

- 问树中有几个联通块的直径介于 $[L,R]$ 之间。 $n \leq 1e5$



一个题

- 显然做差分。求直径介于 $[0, R]$ 的个数- $[0, L-1]$ 的答案。
- 以 $[0, R]$ 为例。显然设 $f[x]$ 为 x 子树内，钦定 x 一定选，并且直径不超过 R 的连通块方案数。可能形成新的直径，发现再记个到 x 的最长链就行了。
- 于是记个 $f[x][h]$ 为为 x 子树内，钦定 x 一定选，到 x 最长链长度为 h 并且直径不超过 R 的方案数。
- 转移： $f[x][\max(hu, hv + 1)] += f[u][hu] * f[v][hv], hu + hv + 1 \leq R$ 。



一个题

- 同样注意到第二维和深度有关，使用长链剖分。
- 同样，先合并重儿子，直接把其f数组拿过来用就好，复杂度 $O(1)$ 。
- 对于轻儿子，直接看转移式子： $f'[u][\max(hu, hv + 1)] += f[u][hu] * f'[v][hv]$, $hu + hv + 1 \leq R$ ，我们需要同时枚举hu和hv。能不能只枚举hv？
- 我们用 $f[x]$ 表示合并前的DP数组，用 $f'[x]$ 表示合并后的。



一个题

- 带着max总是不方便的，我们应该先分类讨论去掉max：
- $hu \leq hv + 1$ ：有 $f'[u][hv + 1] += f[v][hv] * (\sum_{hu+ hv+1 \leq R, du \leq dv+1} f[u][du])$
- 而对于所有满足 $hu > hv + 1$ 的 hu ， $f'[u][du] += f[u][du] * f[v][dv]$ 。
- 第一种转移？区间求和，单点加！
- 第二种转移？区间乘法！
- 维护线段树。



一个题

- 更具体地，为了快速继承重儿子的DP数组，全局只建一棵线段树。在处理u时，就让线段树上 $[dfn[x], dfn[x] + lth[x]]$ 这段区间的值为 $f[x][0] \dots f[x][lth[x]]$ 。
- 还是先递归长儿子。回到父亲时，平移的任务自然就被完成。
- 复杂度 $O(n \log n)$



PKUWC2018 Minimax

题目描述

[M+](#) [复制Markdown](#) [展开](#)

小 C 有一棵 n 个结点的有根树，根是 1 号结点，且每个结点最多有两个子结点。

定义结点 x 的权值为：

- 1.若 x 没有子结点，那么它的权值会在输入里给出，**保证这类点中每个结点的权值互不相同**。
- 2.若 x 有子结点，那么它的权值有 p_x 的概率是它的子结点的权值的最大值，有 $1 - p_x$ 的概率是它的子结点的权值的最小值。

现在小 C 想知道，假设 1 号结点的权值有 m 种可能性，**权值第 i 小的可能性的权值是 V_i** ，它的概率为 $D_i (D_i > 0)$ ，求：

$$\sum_{i=1}^m i \cdot V_i \cdot D_i^2$$

你需要输出答案对 998244353 取模的值。

• $n \leq 3e5$

PKUWC2018 Minimax

- 这是一类经典问题：使用线段树合并维护树形DP。

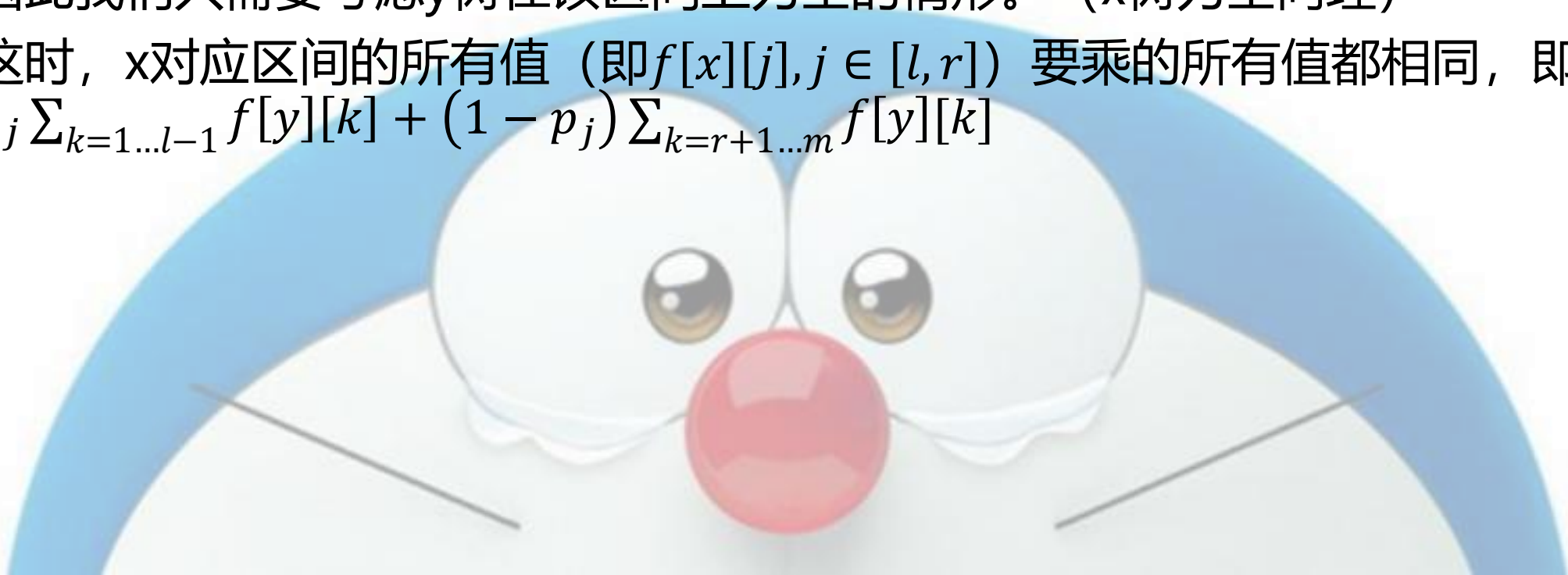


PKUWC2018 Minimax

- 这个式子看起来没啥好的性质，我们得把整个D数组全求出来。
- 显然，把不同权值从小到大排序+离散化后，我们可以搞出这样的DP柿子：
- 设 $f[x][i]$ 为x结点出现第i小权值的概率。设l,r分别为x的左右儿子。
- $f[x][i] = f[l][i] * (p_x \sum_{k=1}^{i-1} f[r][k] + (1 - p_x) \sum_{k=i+1}^m f[r][k]) + f[r][i] * (p_x \sum_{k=1}^{i-1} f[l][k] + (1 - p_x) \sum_{k=i+1}^m f[l][k])$ 。当然实际上 $f[l][i], f[r][i]$ 中有一个必定为0。
- 注意到这个转移柿子是在做：左儿子某个位置j的值乘上（右儿子j前面的前缀和* p +右儿子j后面的后缀和*($1-p$))。

PKUWC2018 Minimax

- 我们用线段树合并维护这个过程。我们现在想把l和r两棵树合并起来扔给x。
- 当我们希望合并[l,r]这个区间。若x和y对应的两棵树在[l,r]都非空，那么递归处理。
- 因此我们只需要考虑y树在该区间上为空的情形。（x树为空同理）
- 这时，x对应区间的所有值（即 $f[x][j], j \in [l, r]$ ）要乘的所有值都相同，即
$$p_j \sum_{k=1 \dots l-1} f[y][k] + (1 - p_j) \sum_{k=r+1 \dots m} f[y][k]$$



PKUWC2018 Minimax

- 我们在线段树合并的过程中顺便维护 $sum1 = \sum_{k=1 \dots l-1} f[y][k]$, $sum2 = \sum_{k=r+1 \dots m} f[y][k]$ 即可, 即当前区间以左的和, 及当前区间以右的和。



PKUWC2018 Minimax

- 我们在线段树合并的过程中顺便维护 $sum1 = \sum_{k=1 \dots l-1} f[y][k]$, $sum2 = \sum_{k=r+1 \dots m} f[y][k]$ 即可, 即当前区间以左的和, 及当前区间以右的和。
- 具体来说, 是向左儿子递归就加上 $sum[ls]*p$, 向右儿子递归就加上 $sum[rs]*(1-p)$ 。
- 对于x树也同样维护的值。



PKUWC2018 Minimax

- 我们在线段树合并的过程中顺便维护 $sum1 = \sum_{k=1 \dots l-1} f[y][k]$, $sum2 = \sum_{k=r+1 \dots m} f[y][k]$ 即可, 即当前区间以左的和, 及当前区间以右的和。
- 具体来说, 是向左儿子递归就加上 $sum[ls]*p$, 向右儿子递归就加上 $sum[rs]*(1-p)$ 。
- 对于x树也同样维护的值。



PKUWC2018 Minimax

```
void merge(int &rt, int rx, int ry, int l, int r, int xmul, int ymul, int pr) { //把rx, ry合并后的线段树扔给rt
    //xmul和ymul分别是rx里的数应该乘的东西和ry里的数应该乘的东西。就是括号里的那一大坨
    //xmul大概是pr*ry的前缀和+(1-pr)*ry的后缀和。ymul pr*rx的前缀和+(1-pr)*rx的后缀和
    if(!rx&&!ry) return rt=0, void();
    if(!rx) {
        color(ry, ymul);
        return rt=ry, void();
    }
    if(!ry) {
        color(rx, xmul);
        return rt=rx, void();
    }
    if(z[rx].col!=1) push_down(rx);
    if(z[ry].col!=1) push_down(ry);
    rt=rx;
    int mid=(l+r)>>1;
    int ryr=z[z[ry].rs].sum, rxr=(z[z[rx].rs].sum), ryl=z[z[ry].ls].sum, rxl=z[z[rx].ls].sum; //必须先记
    merge(z[rt].ls, z[rx].ls, z[ry].ls, l, mid, add(xmul, 111*ryr*(1-pr+p)%p), add(ymul, 111*rxr*(1-pr+p)%p),
    merge(z[rt].rs, z[rx].rs, z[ry].rs, mid+1, r, add(xmul, 111*ryl*pr%p), add(ymul, 111*rxl*pr%p), pr);
    push_up(rt);
}
```

NOI2020命运

题意

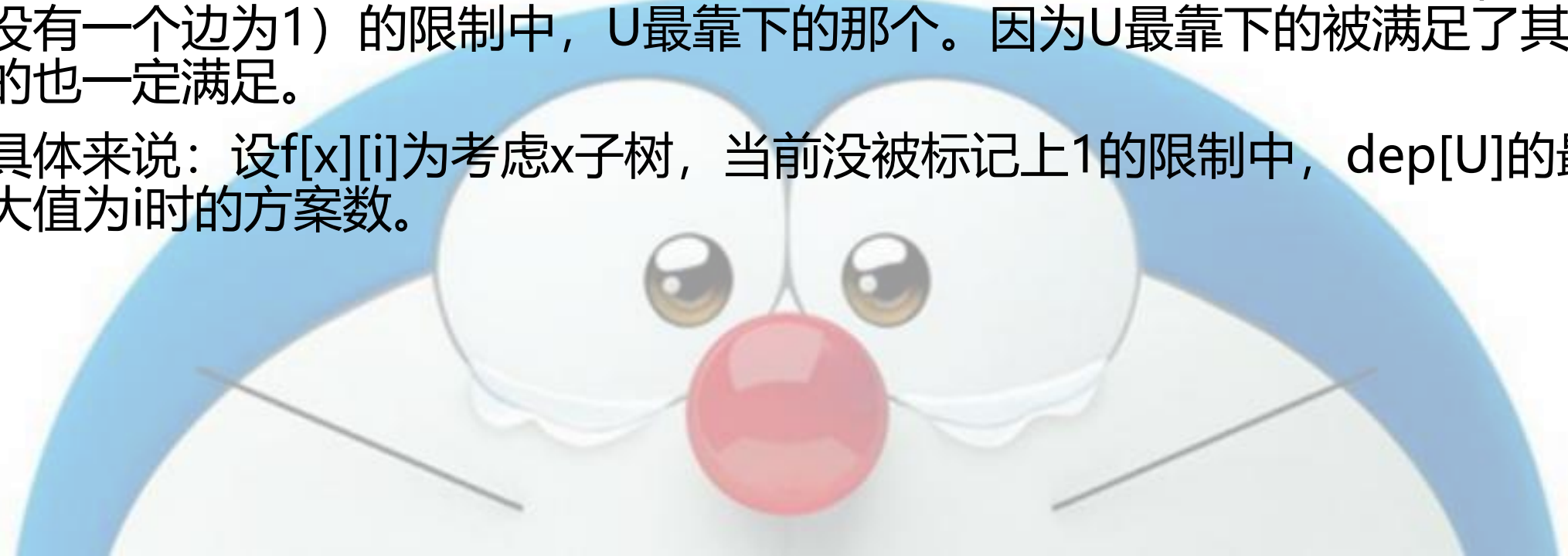
给定一棵 n 个点的树和 m 条限制，你可以给树上的每一条边赋一个 0 或 1 的权值。对于所有限制 (u, v) （保证 v 为 u 的祖先）你需要保证 u 到 v 上至少有一条边的权值为 1，求赋值方案数。

Data Range: $1 \leq n, m \leq 5 \times 10^5$



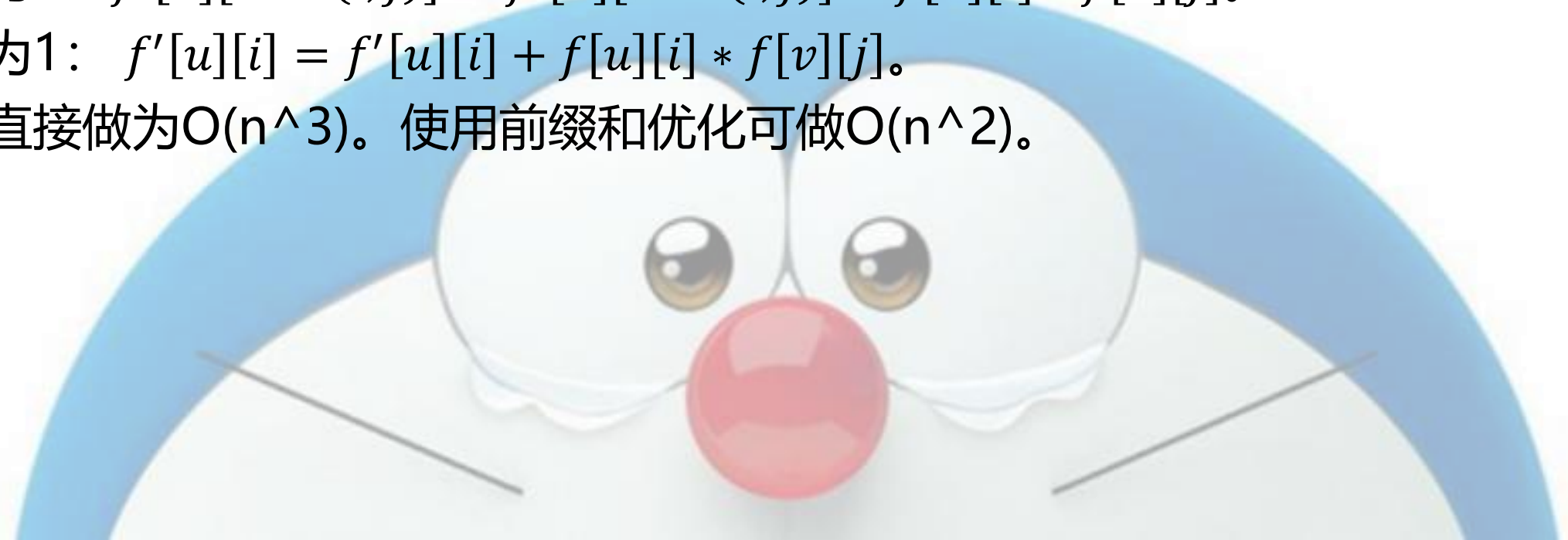
NOI2020命运

- 显然考虑树形DP。
- 子树内的限制一定在子树里处理完了，完全在子树外的限制肯定也和这个子树无关。
- 我们关心的其实是V在x子树内，U在x子树外，当前还没有被标记的（即还没有一个边为1）的限制中，U最靠下的那个。因为U最靠下的被满足了其他的也一定满足。
- 具体来说：设 $f[x][i]$ 为考虑x子树，当前没被标记上1的限制中， $\text{dep}[U]$ 的最大值为i时的方案数。



NOI2020命运

- 转移:
- v 的父亲 u 处考虑 $\langle u, v \rangle$ 这条边标记0还是1。
- 枚举 $i < \text{dep}[x], j < \text{dep}[v]$ 。
- 为0: $f'[u][\max(i, j)] = f'[u][\max(i, j)] + f[u][i] * f[v][j]$ 。
- 为1: $f'[u][i] = f'[u][i] + f[u][i] * f[v][j]$ 。
- 直接做为 $O(n^3)$ 。使用前缀和优化可做 $O(n^2)$ 。



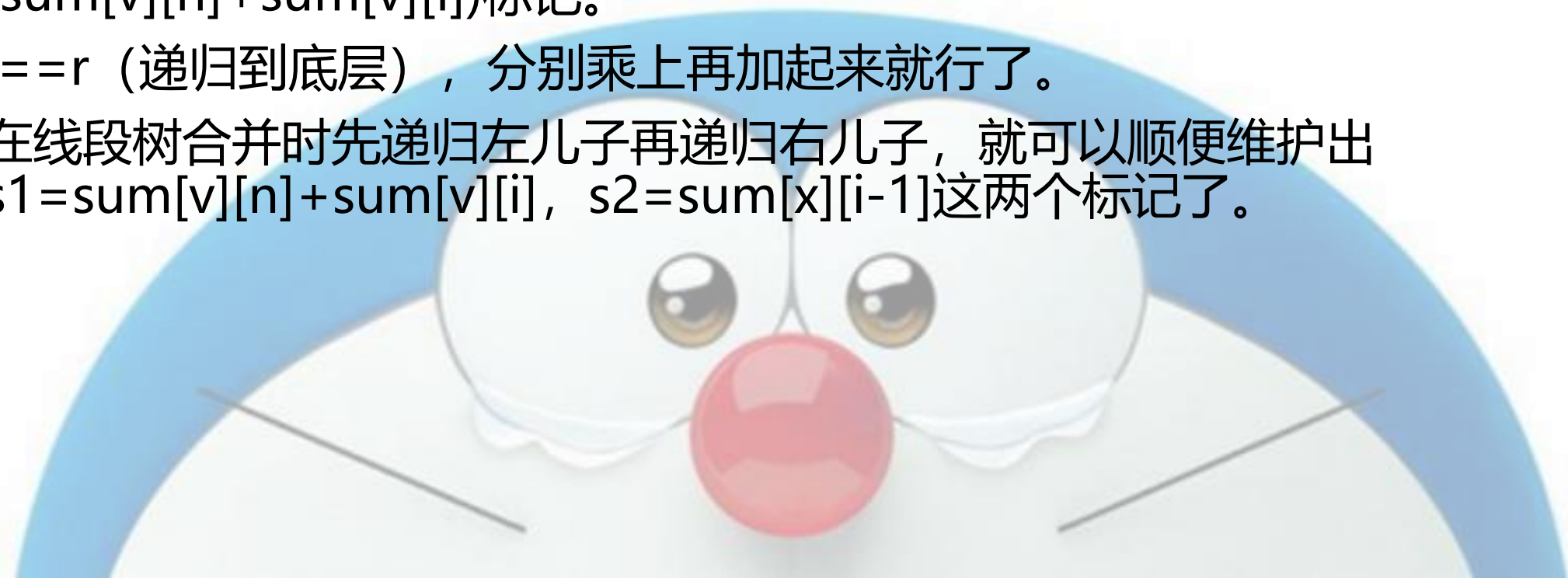
NOI2020命运

- 形式和上题有点像（取max），考虑线段树优化合并。
- 我们改写一下转移（看谁能转移到自己）：
- $f'[u] = f[u][i] * sum[v][n] + f[u][i] * sum[v][i] + f[v][i] * sum[u][i - 1] = f[u][i] * (sum[v][n] + sum[v][i]) + f[v][i] * sum[u][i - 1]$ 。其中sum是f的前缀和。
- 看到前缀和的样子，就和上题很像了。



NOI2020命运

- 只需考虑以下三种情况：
- u 对应节点为空：此时 $f[u][i]=0$ ，应让 v 对应节点打一个乘 $\text{sum}[u][i-1]$ 标记。
- v 对应节点为空：此时 $f[v][i]=0$ ，应让 x 对应节点打一个乘 $(\text{sum}[v][n]+\text{sum}[v][i])$ 标记。
- $l==r$ （递归到底层），分别乘上再加起来就行了。
- 在线段树合并时先递归左儿子再递归右儿子，就可以顺便维护出 $s1=\text{sum}[v][n]+\text{sum}[v][i]$ ， $s2=\text{sum}[x][i-1]$ 这两个标记了。



谢谢大家！

